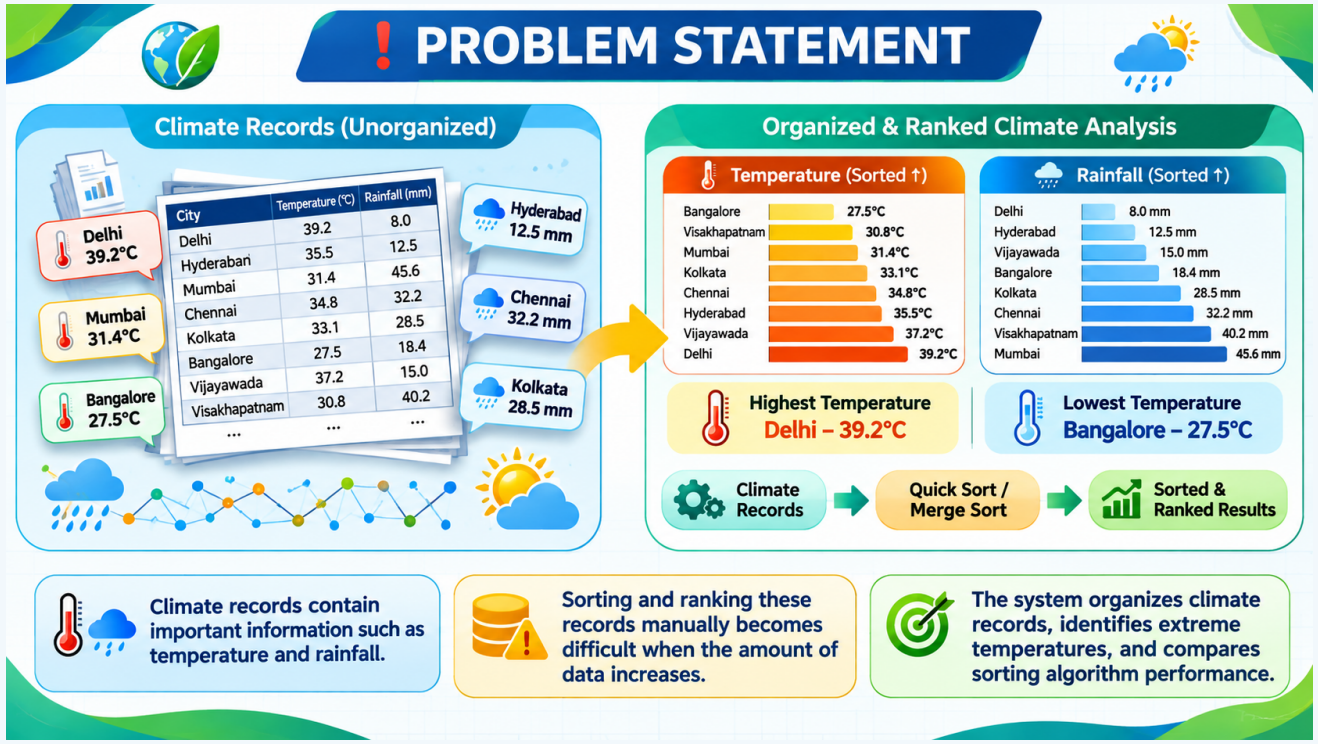

CLIMATE TIME-SERIES
SORTING AND RANKING
Advanced Data Structures (ADS) – C++ Project
Quick Sort • Merge Sort • Climate Data Analysis
SDG 13 – Climate Action
°C

01 | PROBLEM STATEMENT

Project 19 • Environmental and Energy Sciences • SDG 13



THE CHALLENGE

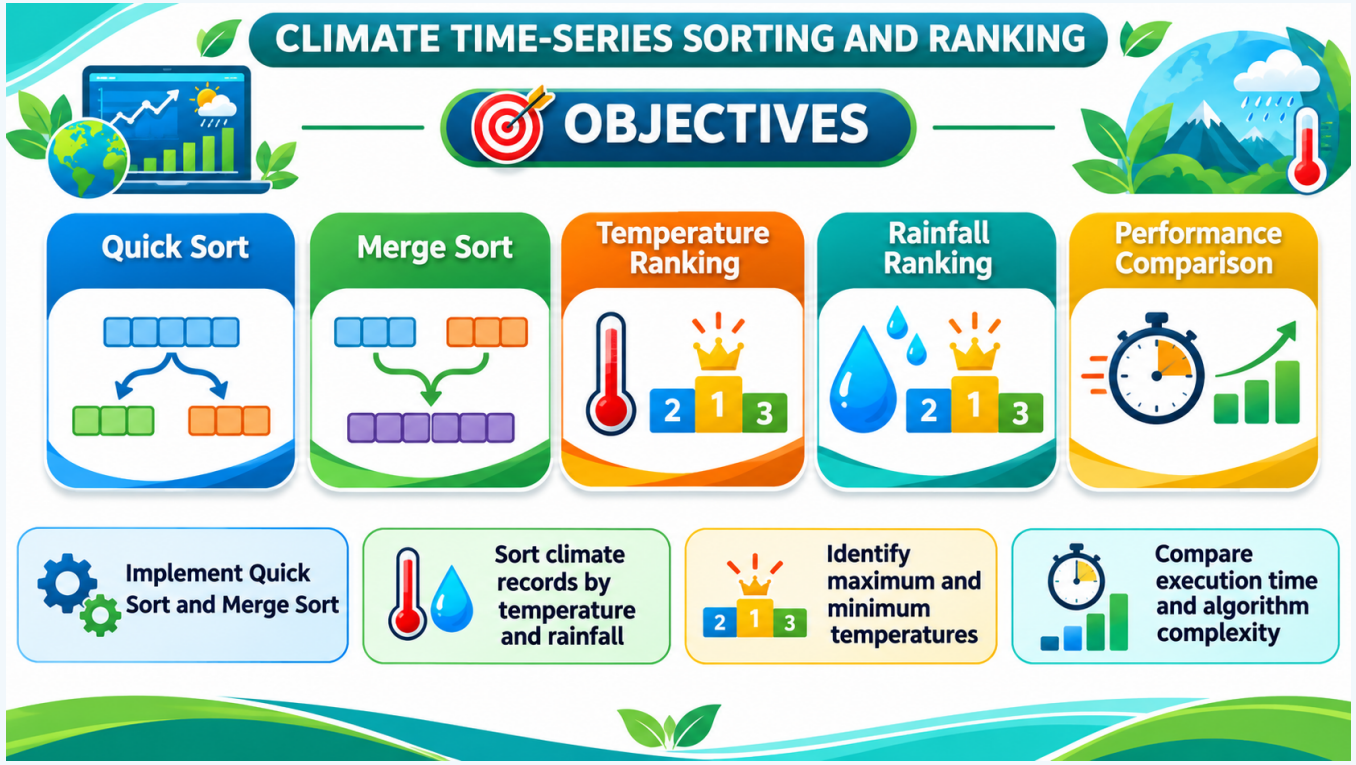
Climate records become harder to organize as datasets grow. Manual so

THE PROPOSED SYSTEM

Use Quick Sort and Merge Sort to order temperature and rainfall records,

02 | PROJECT OBJECTIVES

What the C++ application is designed to demonstrate



SORT CLIMATE RECORDS

Sort by temperature or rainfall.

IDENTIFY EXTREMES

Find the highest and lowest temperatures.

COMPARE PERFORMANCE

Measure Quick Sort and Merge Sort runtime.

03 | DATA MODEL & WORKFLOW

Records are stored in a vector of ClimateRecord structures

CLIMATERECORD

- City name
- Temperature (°C)
- Rainfall (mm)

DATA STORAGE

A C++ vector stores multiple climate records and su

USER OPERATIONS

Display • Sort • Analyze
Compare • Add record

1 Input records

2 Choose field

3 Run algorithm

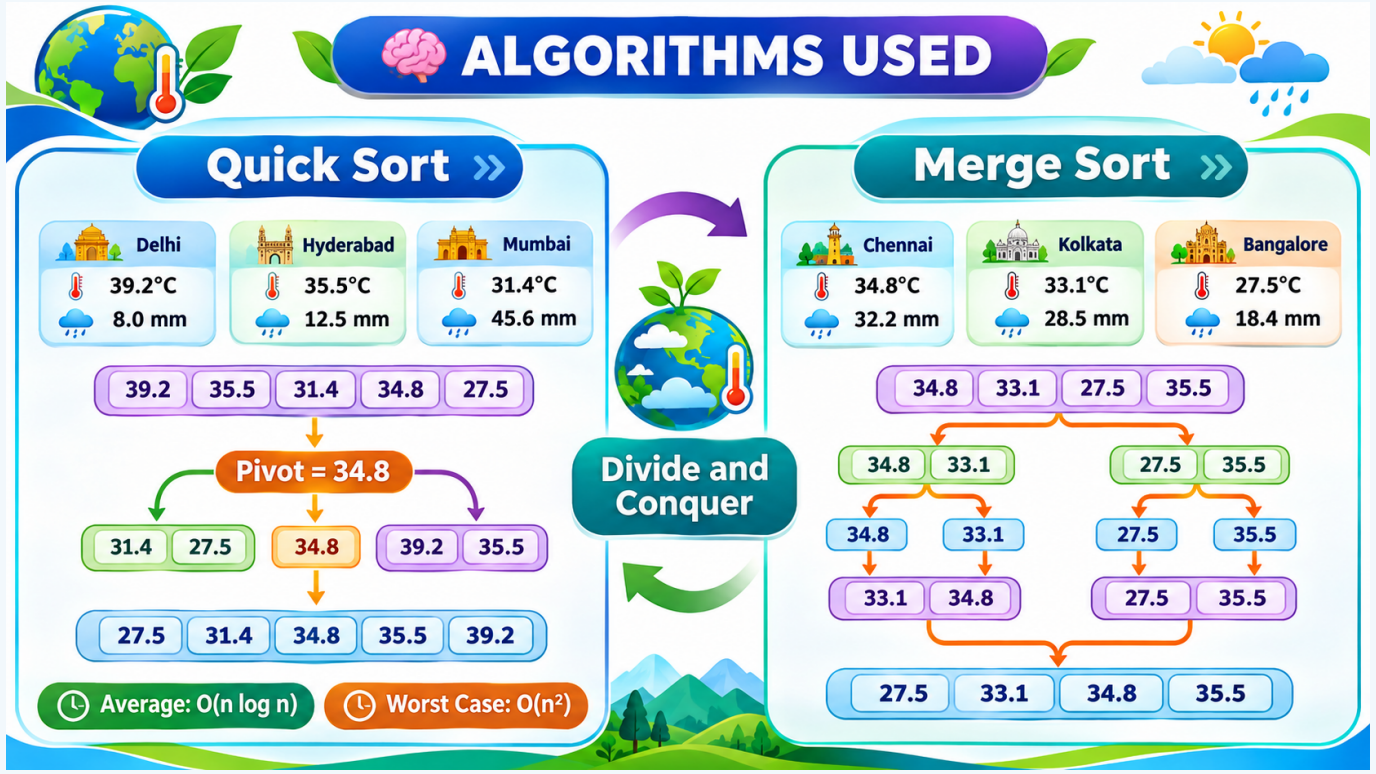
4 View results

5 Compare time

Workflow: initialize data → select operation → sort or analyze → display results.

04 | ALGORITHMS: DIVIDE AND CONQUER

Both algorithms sort the same climate records using different strategies



QUICK SORT

Choose a pivot, partition values around it, then recursively sort the partitions.
Average: O(n log n)
Worst: O(n^2)

MERGE SORT

Divide records into halves, sort each half, then merge the sorted parts.
Best / Average / Worst: O(n log n)

05 | SAMPLE CLIMATE DATA

Illustrative records included in the current C++ program

City	Temperature (°C)	Rainfall (mm)
Hyderabad	35.5	12.5
Delhi	39.2	8.0
Mumbai	31.4	45.6
Chennai	34.8	32.2
Kolkata	33.1	28.5
Bangalore	27.5	18.4
Vijayawada	37.2	15.0
Visakhapatnam	30.8	40.2

DATA FIELDS

City • Temperature • Rainfall

IMPORTANT NOTE

These are sample values for demonstrating the algorithms, not verified live wea

06 | SAMPLE RESULTS

Temperature ranking and extreme-value identification

HIGHEST TEMPERATURE

Delhi
39.2 °C

LOWEST TEMPERATURE

Bangalore
27.5 °C

SORTING OPTIONS

Temperature
Rainfall

Temperature values in ascending order

27.5	30.8	31.4	33.1	34.8	35.5	37.2	39.2
Bangalore	Visakhapatnam	Mumbai	Kolkata	Chennai	Hyderabad	Vijayawada	Delhi

Values shown are based on the built-in sample dataset.

07 | COMPLEXITY & PERFORMANCE

The program measures runtime; values depend on machine and input size

Algorithm	Best	Average	Worst
Quick Sort	O(n log n)	O(n log n)	O(n^2)
Merge Sort	O(n log n)	O(n log n)	O(n log n)

BENCHMARK METHOD

Copy the same records, time each algorithm, and compare elapsed

QUICK SORT

Often fast in practice, but worst-case complexity is O(n^2).

MERGE SORT

Guarantees O(n log n) time, with additional memory for merging.

08 | LIMITATIONS & FUTURE ENHANCEMENTS

Current implementation scope and practical next steps

CURRENT SCOPE

- Built-in sample climate records
- Console-based menu
- Sort by temperature or rainfall
- Quick Sort and Merge Sort
- Highest / lowest temperature
- Runtime comparison

FUTURE ENHANCEMENTS

- Import records from CSV files
- Benchmark larger datasets
- Add charts and visual reports
- Validate user input
- Add more sorting algorithms
- Connect to verified climate datasets

The current program is a learning prototype; future work can extend it for larger and real-world datasets.


CLIMATE TIME-SERIES
SORTING AND RANKING
Advanced Data Structures (ADS) – C++ Project
Quick Sort • Merge Sort • Climate Data Analysis
SDG 13 – Climate Action
°C
THANK YOU
Questions & Discussion